Rapport TP1

Tâche 1 - URLs et GET

Requête GET simple :

```
azdad@Samy MINGW64 ~/OneDrive - Efrei/Documents/INGE2/API_WEB_SERVICES (master)
$ curl.exe -i -s https://jsonplaceholder.typicode.com/posts

HTTP/1.1 200 OK
Date: Mon, 21 Sep 2026 09:22:32 GMT
Content-Type: application/json; charset=utf-8
Transfer-Encoding: chunked
Connection: keep-alive
...

[
  {
    "userId": 1,
    "id": 1,
    "title": "sunt aut facere repellat provident occaecati excepturi optio reprehenderit",
    "body": "quia et suscipit..."
  },
  ...
]
```

- Code de statut : 200 OK (requête réussie).

- Nombre d'éléments : 100 posts.

- Structure : C'est une liste d'objets JSON avec les clés userId, id, title et body.

Paramètres de requête :

```
azdad@Samy MINGW64 ~/OneDrive - Efrei/Documents/INGE2/API_WEB_SERVICES (master)
$ curl.exe -i -s "https://jsonplaceholder.typicode.com/posts?userId=1"
... (10 rÃ©sultats) ...

azdad@Samy MINGW64 ~/OneDrive - Efrei/Documents/INGE2/API_WEB_SERVICES (master)
$ curl.exe -i -s "https://jsonplaceholder.typicode.com/posts?_page=1&_limit=5"
... (5 rÃ©sultats) ...
```

- Résultats : 10 posts avec userId=1, et 5 posts avec _limit=5.

- Intérêt : Ça permet de filtrer ou paginer sans devoir créer une route serveur pour chaque cas.

Tâche 2 - En-têtes HTTP

```
azdad@Samy MINGW64 ~/OneDrive - Efrei/Documents/INGE2/API_WEB_SERVICES (master)
$ curl.exe -i -s -H "User-Agent: MonSuperCurl-TP1" -H "Accept: application/json" https://jsonplaceholder.typicode.com/posts

HTTP/1.1 200 OK
Date: Mon, 21 Sep 2026 09:25:18 GMT
Content-Type: application/json; charset=utf-8
...
```

- L'en-tête de requête est envoyé par le client pour dire ce qu'il veut, l'en-tête de réponse est renvoyé par le serveur pour décrire ce qu'il donne.

- Accept = ce que le client veut recevoir. Content-Type = le format réel envoyé.

Tâche 3 - POST, PUT, PATCH, DELETE

Créer (POST) :

```
azdad@Samy MINGW64 ~/OneDrive - Efrei/Documents/INGE2/API_WEB_SERVICES (master)
$ curl.exe -i -s -X POST -H "Content-Type: application/json" -d @new_post.json https://jsonplaceholder.typicode.com/posts

HTTP/1.1 201 Created
Date: Mon, 21 Sep 2026 09:30:00 GMT
Content-Type: application/json; charset=utf-8
...

{
  "title": "Nouveau post",
  "body": "Ceci est le corps du nouveau post.",
  "userId": 1,
  "id": 101
}
```

Remplacer (PUT) :

```
azdad@Samy MINGW64 ~/OneDrive - Efrei/Documents/INGE2/API_WEB_SERVICES (master)
$ curl.exe -i -s -X PUT -H "Content-Type: application/json" -d @update_post.json https://jsonplaceholder.typicode.com/posts/1

HTTP/1.1 200 OK
...

{
  "id": 1,
  "title": "Post mis Ã  jour",
  "body": "Ce post a Ã©tÃ© mis Ã  jour.",
  "userId": 1
}
```

Modifier (PATCH) :

```
azdad@Samy MINGW64 ~/OneDrive - Efrei/Documents/INGE2/API_WEB_SERVICES (master)
$ curl.exe -i -s -X PATCH -H "Content-Type: application/json" -d @patch_post.json https://jsonplaceholder.typicode.com/posts/1

HTTP/1.1 200 OK
...

{
  "userId": 1,
  "id": 1,
  "title": "Titre modifiÃ© seulement",
  "body": "quia et suscipit\nsuscipit recusandae consequuntur expedita..."
}
```

- PUT remplace toute la ressource, PATCH modifie juste les champs donnés (ici le titre est modifié et le reste est gardé).

Supprimer (DELETE) :

```
azdad@Samy MINGW64 ~/OneDrive - Efrei/Documents/INGE2/API_WEB_SERVICES (master)
$ curl.exe -i -s -X DELETE https://jsonplaceholder.typicode.com/posts/1

HTTP/1.1 200 OK
Date: Mon, 21 Sep 2026 09:30:08 GMT
Content-Type: application/json; charset=utf-8
...

{}
```

- La réponse 204 No Content a un corps vide. C'est normal car la ressource est supprimée, le serveur n'a plus rien à renvoyer à part confirmer le succès.

Tâche 4 - Erreurs

404 Not Found :

```
azdad@Samy MINGW64 ~/OneDrive - Efrei/Documents/INGE2/API_WEB_SERVICES (master)
$ curl.exe -i -s https://jsonplaceholder.typicode.com/posts/9999

HTTP/1.1 404 Not Found
Date: Mon, 21 Sep 2026 09:35:10 GMT
Content-Type: application/json; charset=utf-8
...

{}
```

405 Method Not Allowed :

```
azdad@Samy MINGW64 ~/OneDrive - Efrei/Documents/INGE2/API_WEB_SERVICES (master)
$ curl.exe -i -s -X FOO https://jsonplaceholder.typicode.com/posts/1

HTTP/1.1 405 Method Not Allowed
Date: Mon, 21 Sep 2026 09:35:20 GMT
Content-Type: application/json; charset=utf-8
...

{
  "error": "Method Not Allowed"
}
```

400 Bad Request :

```
azdad@Samy MINGW64 ~/OneDrive - Efrei/Documents/INGE2/API_WEB_SERVICES (master)
$ curl.exe -i -s -X POST -H "Content-Type: application/json" -d "{"title": "Test",}" https://jsonplaceholder.typicode.com/posts

HTTP/1.1 400 Bad Request
Date: Mon, 21 Sep 2026 09:35:30 GMT
Content-Type: text/html; charset=utf-8
...

SyntaxError: Unexpected token } in JSON at position 16
```

- Dans ces 3 cas (classe 4xx), l'erreur vient du client car il s'est trompé dans l'URL, la méthode ou le JSON envoyé. Les erreurs du serveur sont en 5xx.

Tâche 5 - Tests

```
azdad@Samy MINGW64 ~/OneDrive - Efrei/Documents/INGE2/API_WEB_SERVICES (master)
$ ./tests.ps1
Running Tests for GET /posts/1...
  [PASS] Le code de statut est 200
  [PASS] La reponse contient un champ id
  [PASS] Le temps de reponse est inferieur a 1000 ms

Running Tests for POST /posts...
  [PASS] Le code de statut est 201
  [PASS] La reponse contient un champ id
```

- C'est plus fiable qu'à l'oeil nu car ça évite les erreurs d'inattention et c'est automatique.

- C'est indispensable en intégration continue (CI/CD) pour vérifier qu'on ne casse rien avant une mise en prod.

Tâche 6 - Jeton

```
azdad@Samy MINGW64 ~/OneDrive - Efrei/Documents/INGE2/API_WEB_SERVICES (master)
$ curl.exe -i -s -H "Authorization: Bearer demo-token-12345" https://jsonplaceholder.typicode.com/posts/1

HTTP/1.1 200 OK
Date: Mon, 21 Sep 2026 09:40:12 GMT
Content-Type: application/json; charset=utf-8
...
```

- Syntaxe : Authorization: Bearer <jeton>

- Le HTTPS est obligatoire pour chiffrer le token, sinon il passe en clair sur le réseau et peut être volé.

Tâche 7 - Documentation

- La doc Postman sert de contrat d'API : elle liste les routes et les formats attendus pour que le client puisse l'utiliser sans regarder le code serveur.

Récapitulatif et Réflexion

GET /posts -> 200 OK (Liste des posts récupérée)

POST /posts -> 201 Created (Post créé)

PUT /posts/1 -> 200 OK (Post remplacé)

PATCH /posts/1 -> 200 OK (Titre modifié)

DELETE /posts/1 -> 200 OK (Post supprimé)

Le protocole HTTP permet de faire des API propres et uniformes grâce à ses méthodes (GET, POST...) et ses codes de retour clairs (200, 404...). Ça rend le système logique.

L'ajout de tests automatisés (Tâche 5) permet de sécuriser l'application. Sur un projet, on ne peut pas tester chaque route à la main à chaque changement. Les tests s'assurent que les modifications de code ne créent pas de bugs sur l'API existante.